

Login simples com PocketBase e Alpine.js

Construa duas páginas, entenda cada tecnologia e depois repita o projeto sem acompanhar o professor.

A pergunta do projeto

Como saber se quem abriu a segunda página realmente informou uma conta válida? Um link não responde a isso. Vamos avançar até a confirmação no servidor.

Quatro etapas, uma ideia nova por vez

ETAPA	IDEIA NOVA	BRANCH
01	Arquivos separados e duas páginas	etapa-01-html-estatico
02	Alpine mostra o e-mail digitado	etapa-02-alpine
03	PocketBase autentica a conta	etapa-03-login-pocketbase
04	Confirmar sessão e sair	etapa-04-sessao-e-saida

Como estudar

Leia a explicação, digite o trecho de código, faça o experimento e responda à pergunta de cada etapa. As branches mostram o resultado esperado quando você precisar comparar.

Público: alunos iniciantes em HTML, CSS e JavaScript.

Entenda o problema antes do código

Uma escola quer uma tela de entrada e uma área do aluno. O navegador desenha as telas; o PocketBase guarda as contas e decide se e-mail e senha correspondem a um usuário cadastrado.

```
aluno -> login.html -> Alpine.js -> PocketBase
      <- resposta
      -> logado.html (após sucesso)
```

Quem faz o quê?

- HTML organiza títulos, campos, botões e links.
- CSS define aparência e espaçamento.
- JavaScript executa as ações programadas.
- Alpine.js liga valores do JavaScript aos elementos HTML.
- PocketBase confere as credenciais no servidor e emite a sessão.

Três ideias que parecem iguais, mas não são

- Navegar: abrir outro arquivo por um link. Qualquer pessoa pode fazê-lo.
- Reagir: atualizar o texto da página quando algo é digitado. Isso ocorre no navegador.
- Autenticar: pedir ao servidor para verificar uma conta.

O que você vai produzir

Desde o começo haverá login.html, logado.html, css/estilo.css, js/login.js e js/logado.js. O arquivo js/pocketbase.js será criado na etapa 3.

Depois da explicação do professor, use as páginas 11 e 12 para reconstruir tudo sozinho.

Prepare seu ambiente

Você precisa de um editor, navegador, Python para servir os arquivos e PocketBase para as etapas 3 e 4. As bibliotecas Alpine.js e SDK do PocketBase são carregadas por CDN; mantenha internet ativa.

1. Crie a pasta do projeto

```
login-simples/  
  login.html  
  logado.html  
  css/estilo.css  
  js/login.js  
  js/logado.js
```

2. Sirva os arquivos no navegador

Abra um terminal dentro de login-simples e execute:

```
py -m http.server 5500  
# alternativa: python -m http.server 5500
```

Acesse <http://127.0.0.1:5500/login.html>. Não feche esse terminal enquanto estiver trabalhando.

3. Prepare o PocketBase antes da etapa 3

- Baixe e extraia o PocketBase pelo site oficial: pocketbase.io.
- Em outro terminal, na pasta do executável, rode o comando abaixo.
- Abra http://127.0.0.1:8090/_/ e crie o administrador.
- Crie uma coleção do tipo Auth chamada users e um aluno fictício com e-mail e senha.

```
.\pocketbase.exe serve
```

Dois servidores

A interface usa a porta 5500. O PocketBase usa a porta 8090. O navegador chama o PocketBase quando o aluno envia o formulário.

Etapa 1 | Monte as duas páginas

Branch de referência: etapa-01-html-estatico

Crie os cinco arquivos mostrados na página anterior. Nesta etapa, os JS podem conter apenas um comentário. Isso reserva o lugar do comportamento sem inventar uma função desnecessária.

Comece por login.html

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Login dos alunos</title>
  <link rel="stylesheet" href="css/estilo.css">
  <script defer src="js/login.js"></script>
</head>
<body>
  <main class="cartao">
    <h1>Entrar</h1>
    <label for="email">E-mail</label>
    <input id="email" type="email">
    <label for="senha">Senha</label>
    <input id="senha" type="password">
    <button type="button" disabled>Entrar (em breve)</button>
    <p><a href="logado.html">Conhecer a segunda página</a></p>
  </main>
</body>
</html>
```

Em logado.html, use a mesma estrutura head. Troque o título e o conteúdo de main por uma mensagem: 'Você chegou por um link; ainda não existe login'. Acrescente um link de volta para login.html e carregue js/logado.js.

Experimento

Abra logado.html sem preencher os campos. Explique por que isso foi possível. O botão está desativado; foi o link que abriu a página.

Etapa 1 | Dê forma e entenda os caminhos

Escreva em `css/estilo.css` um estilo pequeno. Você pode ampliar as cores e espaçamentos depois, sem alterar o funcionamento do login.

```
:root {
  font-family: Arial, sans-serif;
  color: #1f2937;
  background: #f3f4f6;
}
* { box-sizing: border-box; }
body {
  min-height: 100vh;
  margin: 0;
  display: grid;
  place-items: center;
}
.cartao {
  width: min(100%, 380px);
  padding: 32px;
  background: white;
  border: 1px solid #e5e7eb;
}
label { display: block; margin-top: 16px; }
input { width: 100%; padding: 10px; }
button { margin-top: 20px; }
```

No HTML, o caminho `css/estilo.css` começa na mesma pasta do HTML. O navegador procura a pasta `css` e depois o arquivo `estilo.css`. O mesmo raciocínio vale para `js/login.js`.

Confira sua etapa

- As duas páginas têm conteúdo e estilos.
- Mudar a cor em `css/estilo.css` altera as duas páginas.
- O botão Entrar não faz nada; o link abre a segunda página.
- `js/login.js` e `js/logado.js` existem, mas ainda não executam ações.

Pergunta de compreensão

Se você apagar o link e mantiver os campos, alguém consegue chegar à segunda página digitando e-mail e senha? Por quê?

Etapa 2 | Veja Alpine.js reagir

Branch de referência: etapa-02-alpine

Alpine.js vai copiar para a tela o e-mail enquanto você digita. Não há envio nem autenticação. O botão continua desativado.

1. Carregue a biblioteca depois do seu JS

```
<script defer src="js/login.js"></script>
<script defer src="https://cdn.jsdelivr.net/npm/alpinejs@3.17.4/dist/cdn.min.js"></script>
```

A ordem importa: sua função precisa existir quando Alpine iniciar.

2. Acrescente os atributos ao HTML

```
<main class="cartao" x-data="formulario()" x-cloak>
  <label for="email">E-mail</label>
  <input id="email" type="email" x-model="email">
  <p x-show="email">E-mail digitado:
    <strong x-text="email"></strong>
  </p>
  <!-- Mantenha o campo de senha e o botão desativado. -->
</main>
```

3. Escreva js/login.js

```
function formulario() {
  return { email: '' };
}
```

x-data cria o estado; x-model atualiza email; x-text exibe esse valor; x-show esconde o parágrafo quando ele está vazio. Em CSS, adicione [x-cloak] { display: none !important; } para evitar texto incompleto antes de Alpine iniciar.

Experimente

Digite e apague um e-mail. A tela muda imediatamente. Verifique que logado.html ainda pode ser aberta pelo link e que nenhuma senha foi enviada.

Etapa 3 | Conecte o PocketBase

Branch de referência: etapa-03-login-pocketbase

Só agora o botão Entrar ganha uma ação. Antes, confirme que o PocketBase está rodando e que a coleção Auth users contém um aluno.

1. Crie js/pocketbase.js

```
window.pb = new PocketBase('http://127.0.0.1:8090');
```

2. Em login.html, carregue os scripts nesta ordem

```
<script defer src="https://cdn.jsdelivr.net/npm/pocketbase@0.28.1/dist/pocketbase.umd.js"></script>
<script defer src="js/pocketbase.js"></script>
<script defer src="js/login.js"></script>
<script defer src="https://cdn.jsdelivr.net/npm/alpinejs@3.17.4/dist/cdn.min.js"></script>
```

O SDK define PocketBase; o arquivo de configuração cria window.pb; js/login.js define a função; Alpine lê x-data por último.

3. Transforme os campos em formulário

```
<main class="cartao" x-data="loginComPocketBase()">
  <form @submit.prevent="entrar">
    <input type="email" x-model="email" required>
    <input type="password" x-model="senha" required>
    <p x-show="erro" x-text="erro" role="alert"></p>
    <button type="submit" :disabled="carregando">
      <span x-text="carregando ? 'Entrando...' : 'Entrar'"></span>
    </button>
  </form>
</main>
```

Mantenha as labels dos dois campos. @submit.prevent executa entrar() e evita o recarregamento automático da página.

Etapa 3 | Faça o login real

Substitua o conteúdo de `js/login.js`. A função devolve o estado inicial e a ação de entrar.

```
function loginComPocketBase() {
  return {
    email: '', senha: '', erro: '', carregando: false,
    async entrar() {
      this.erro = '';
      this.carregando = true;
      try {
        await window.pb.collection('users')
          .authWithPassword(this.email, this.senha);
        window.location.href = 'logado.html';
      } catch (erro) {
        this.erro = 'Confira seus dados e o PocketBase.';
      } finally {
        this.carregando = false;
      }
    },
  };
}
```

Leia o caminho de execução

- `async/await` espera o servidor responder antes de abrir outra página.
- `authWithPassword` envia e-mail e senha para a coleção `Auth users`.
- `try` é o caminho do sucesso; `catch` trata erro; `finally` encerra a espera nos dois casos.
- O SDK guarda o token de autenticação no navegador após o sucesso.

Experimento

Tente uma senha errada: continue na tela de login. Tente a correta: abra `logado.html`. Depois abra `logado.html` diretamente em outra janela; nesta etapa ela ainda não confirma a sessão.

O que significa token?

É uma credencial emitida pelo servidor depois do login. Em pedidos posteriores, o SDK a envia ao PocketBase. Guardar um token no navegador não prova, sozinho, que a sessão ainda será aceita pelo servidor.

Etapa 4 | Confirme a sessão

Branch de referência: etapa-04-sessao-e-saida; versão final: main

Agora a segunda página pergunta ao PocketBase se a sessão ainda vale. Só depois mostra o usuário.

Em `js/logado.js`, crie a função da página

```
function paginaLogada() {
  return {
    usuario: null,
    verificando: true,
    async init() {
      if (!window.pb.authStore.isValid) {
        window.location.replace('login.html');
        return;
      }
      try {
        await window.pb.collection('users').authRefresh();
        this.usuario = window.pb.authStore.record;
      } catch (erro) {
        window.pb.authStore.clear();
        window.location.replace('login.html');
      } finally {
        this.verificando = false;
      }
    },
    sair() {
      window.pb.authStore.clear();
      window.location.replace('login.html');
    },
  };
}
```

`init()` é chamado quando Alpine inicia este componente. `isValid` faz apenas uma checagem local do prazo do token; `authRefresh` consulta o servidor. `clear()` apaga o estado de login do SDK.

Etapa 4 | Mostre o usuário e permita sair

Em `logado.html`, carregue os scripts na ordem: SDK do PocketBase, `js/pocketbase.js`, `js/logado.js` e `Alpine.js`. Use as mesmas URLs da etapa 3.

No corpo da página

```
<main class="cartao" x-data="paginaLogada()" x-cloak>
  <p x-show="verificando">Verificando login...</p>
  <section x-show="usuario">
    <h1>Área do aluno</h1>
    <p>Você entrou com:</p>
    <strong x-text="usuario && usuario.email"></strong>
    <button type="button" @click="sair">Sair</button>
  </section>
</main>
```

Enquanto verificando é verdadeiro, a página informa que está esperando. `usuario` começa vazio; após `authRefresh`, recebe o registro confirmado pelo servidor.

Onde os dados privados são protegidos?

Qualquer pessoa pode baixar HTML, CSS e JS. O redirecionamento melhora o uso da página, mas dados privados precisam de regras de API no PocketBase. Em uma coleção com registros privados, uma regra de leitura para contas autenticadas pode ser:

```
@request.auth.id != ""
```

Faça o teste

Entre, atualize `logado.html`, clique em Sair e tente abrir a página diretamente. Depois repita em uma janela anônima. Explique em cada caso o que o navegador sabe e o que o servidor confirma.

Agora reconstrua sem olhar o código pronto

Feche as branches de referência. Em uma pasta vazia, use este roteiro. Só consulte a branch correspondente depois de tentar localizar o problema.

Lista de construção

- 1. Crie login.html, logado.html, css/estilo.css, js/login.js e js/logado.js.
- 2. Ligue o CSS e os JS ao HTML. Faça o link entre as páginas. Deixe Entrar desativado.
- 3. Sirva os arquivos pela porta 5500. Observe os estilos nas duas páginas.
- 4. Carregue Alpine.js, defina formulario() em js/login.js e mostre o e-mail digitado com x-model e x-text.
- 5. Inicie o PocketBase, crie users do tipo Auth e cadastre um aluno fictício.
- 6. Crie js/pocketbase.js. Carregue SDK, configuração, JS da página e Alpine nessa ordem.
- 7. Ative o formulário e implemente authWithPassword com try/catch/finally.
- 8. Em logado.html, carregue os scripts e confirme a sessão com authRefresh antes de mostrar o e-mail.
- 9. Implemente Sair com authStore.clear() e retorno ao login.

Critérios para dizer que terminou

- Senha incorreta mostra erro sem abrir a área do aluno.
- Senha correta abre a área e mostra o e-mail cadastrado.
- Atualizar a página confirma a sessão novamente.
- Sair limpa a sessão; acesso direto sem login volta ao formulário.
- Você consegue explicar o papel de cada arquivo sem ler comentários.

Se travar

Compare primeiro caminhos de arquivos e ordem dos scripts. Depois confira a coleção users, o endereço do PocketBase e o console do navegador.

Diagnóstico e próximos passos

Quando algo falhar, investigue uma causa por vez.

- Sem estilos: confira o caminho `css/estilo.css` no link do HTML.
- O e-mail não aparece na etapa 2: confira `x-data`, `x-model`, `x-text` e se a CDN carregou.
- PocketBase não responde: abra `http://127.0.0.1:8090/_/` e confira `js/pocketbase.js`.
- Credenciais falham: confira se `users` é `Auth` e se o aluno foi cadastrado nela.
- A tela fica vazia: confira erros no console e a ordem dos scripts `defer`.
- O login parece sumir: use sempre o mesmo endereço `127.0.0.1` no navegador.

Glossário curto

- Estado: valores usados pela interface, como email ou carregando.
- API: forma de um programa pedir uma ação ou dados a outro.
- Autenticação: conferir quem está tentando entrar.
- Sessão: estado de login mantido após a autenticação.
- Token: credencial emitida pelo servidor para pedidos posteriores.
- Regra de API: condição que o servidor aplica antes de devolver dados.

Desafios depois de concluir

- Mude a cor do cartão e identifique qual arquivo controla esse visual.
- Mostre um nome cadastrado no registro do aluno, além do e-mail.
- Crie uma coleção de exemplo e restrinja sua leitura a usuários autenticados.

Documentação oficial

[Alpine.js: x-model](#)

[PocketBase: autenticação](#)

[PocketBase: regras de API](#)